

ParkEase — Antigravity Master Development Prompt

MASTER DEVELOPMENT PROMPT — PARKEASE SMART PARKING SYSTEM

You are working on my existing college major project called PARKEASE — Smart Parking System.

IMPORTANT:

Do NOT blindly rewrite the project from scratch.

First inspect the COMPLETE existing repository, understand the current architecture, database models, APIs, frontend, ESP32 code, environment variables, and current implementation.

Then make the required changes coherently across the project.

The existing working functionality must NOT be broken while adding the new features.

1. PROJECT OBJECTIVE

ParkEase is an IoT-based smart parking management system.

Users must be able to:

- Register and login
- View live parking slot availability
- Book an available parking slot
- View their booking
- Enter parking using an authorized booking
- Physically control the parking gate using ESP32 + servo
- Detect vehicles using IR sensors
- Detect parking slot occupancy
- Exit the parking area
- Cancel bookings

Admin must be able to:

- View live parking status
- View all bookings
- View active vehicles
- View parking statistics
- View sensor/gate status

2. ARCHITECTURE

Current:

User → Frontend → Flask REST API → MySQL
ESP32 → Wi-Fi → Flask REST API → MySQL

Target deployment:

Frontend → Vercel
Backend → Render or Railway
Database → MySQL-compatible cloud database
ESP32 → deployed Flask API

ESP32 must NEVER connect directly to MySQL.
Backend/database is authoritative.

3. HARDWARE

Keep GPIO assignments unchanged:

Entry IR → GPIO18
Exit IR → GPIO19
Slot 1 IR → GPIO21
Slot 2 IR → GPIO22
Slot 3 IR → GPIO23
Servo signal → GPIO25

IR sensors are ACTIVE LOW.
Servo uses external 5V and common GND.
Hardware is already tested and working.

4. PHYSICAL LAYOUT

Road / Car
↓
Entry IR
↓
Servo Gate
↓
Exit IR
↓
Parking Area
↓
S1 / S2 / S3

5. ENTRY FLOW

1. User logs in.
2. Views available slots.
3. Books a slot.
4. Backend reserves it.
5. Vehicle reaches entrance.
6. Entry IR detects waiting vehicle.
7. User presses ENTER PARKING.
8. Backend verifies authenticated user, active booking, correct vehicle, reserved slot, and unused entry authorization.
9. Backend authorizes entry.
10. ESP32 receives authorization.
11. Servo opens.
12. Vehicle crosses gate.
13. Exit IR detects vehicle.
14. ESP32 reports entry-crossed.
15. Backend changes ENTERING → IDLE.
16. Gate closes.
17. Slot sensor detects vehicle.
18. Reserved slot becomes OCCUPIED.

Handle timing correctly so a slot report arriving after ENTERING → IDLE still changes the correct RESERVED slot to OCCUPIED.

6. EXIT FLOW

1. Vehicle approaches exit.
2. Exit IR detects it.
3. ESP32 requests exit.
4. Backend identifies an OCCUPIED slot that becomes empty.
5. Slot becomes EXITING.
6. It must not immediately become AVAILABLE.
7. Gate opens.
8. Vehicle passes.
9. Entry IR confirms vehicle fully exited.
10. ESP32 reports completion.
11. EXITING → AVAILABLE.
12. Gate closes.

With only three slot sensors, exact vehicle-to-slot mapping cannot always be known. For the prototype, use the occupied slot whose sensor changes OCCUPIED → EMPTY during active exit.

7. EXISTING APIS

Inspect and preserve existing functionality:

```
GET /api/slots
POST /api/book
POST /api/cancel
POST /api/enter-parking
POST /api/iot/entry
GET /api/iot/entry-status
GET /api/entry-status
POST /api/iot/slots
POST /api/iot/exit
POST /api/iot/entry-crossed
```

Do not remove working endpoints.

8. STATE MACHINE

Movement:

IDLE, ENTERING, EXITING

Slots:

AVAILABLE, RESERVED, OCCUPIED, EXITING

Sensors:

EMPTY, OCCUPIED

Expected:

AVAILABLE → RESERVED → OCCUPIED → EXITING → AVAILABLE

Entry: IDLE → ENTERING → IDLE

Exit: IDLE → EXITING → IDLE

9. BOOKING CONCURRENCY

Only one user may successfully book a slot when concurrent requests occur.

Use proper transactions/row locking.

The second user should receive a friendly "Slot no longer available" message.

Do not hide all IntegrityErrors behind a generic message. Log the real server-side cause safely and return an appropriate user-facing message.

Inspect the actual schema before changing booking logic.

10. USER AUTHENTICATION

Add:

REGISTER:

- Full name
- Email
- Mobile number
- Password
- Confirm password

LOGIN:

- Email
- Password

Use secure password hashing.

Never store plain-text passwords.

Prevent duplicate emails.

Implement proper session/token authentication.

Provide Register, Login, Logout and My Account.

Do not rely only on localStorage for authentication.

11. USER BOOKING

After login:

- View slots
- Select slot
- Enter vehicle number
- Book
- View booking
- Cancel
- Enter parking

Booking should contain appropriate:

Booking ID, user, vehicle, slot, status, booking time, entry time, exit time.

Users must only see/access their own bookings.

12. ENTER PARKING UI

ENTER PARKING should require:

- Logged-in user
- Active booking
- Valid reserved slot
- Entry not already completed

Backend must verify everything.

Show useful status messages such as:

Waiting for vehicle

Entry authorized

Gate opening

Vehicle entering

Parking confirmed

13. ADMIN AUTHENTICATION

Add ADMIN LOGIN only.

No public admin registration.

Admin email/password must be configured server-side using environment variables such as:

ADMIN_EMAIL

ADMIN_PASSWORD or preferably ADMIN_PASSWORD_HASH

Never hardcode admin credentials in frontend JavaScript or expose them in source control.

14. ADMIN DASHBOARD

Create a separate protected admin dashboard.

Overview cards:

- Total Slots
- Available Slots
- Reserved Slots
- Occupied Slots
- Exiting Slots
- Active Bookings
- Active Vehicles
- Today's Bookings
- Today's Completed Exits

Live Parking:

- S1, S2, S3
- Status
- Sensor status
- Vehicle if available
- Automatic refresh

All Bookings:

- Booking ID
- User
- Email
- Mobile
- Vehicle
- Slot
- Status
- Entry Time
- Exit Time
- Created At
- Search/filter/sort
- Responsive table
- Pagination if useful

Active Vehicles:

- Vehicle number
- User
- Slot
- Entry time
- Current status

Statistics:

- Total bookings
- Available vs occupied
- Daily bookings
- Completed sessions
- Active vehicles
- Slot utilization

Use useful charts only.

Sensor/Gate status:

- Entry sensor
- Exit sensor
- S1/S2/S3 sensors
- Gate OPEN/CLOSED
- Movement IDLE/ENTERING/EXITING
- Pending entry booking
- Pending exit slot
- Last IoT update if available

15. ADMIN UI

Professional desktop sidebar + main content.

Responsive collapsible sidebar/hamburger menu on mobile.

16. FRONTEND UI

Preserve ParkEase branding and existing functionality, but make the UI modern, beautiful and professional.

Improve:

- Typography
- Spacing
- Cards
- Buttons
- Shadows
- Navigation
- Parking visualization
- Status badges
- Empty/loading/error/success states
- Smooth transitions

17. RESPONSIVE DESIGN

Must work on:

320px, 375px, 390px, 430px, 768px, 1024px, 1440px+.

Prevent:

- horizontal overflow
- broken tables
- overlapping buttons
- broken navbar
- oversized modals
- inaccessible controls

Mobile booking must be especially good.

18. ANIMATIONS

Add tasteful:

- Page transitions
- Card hover
- Button hover
- Slot status transitions
- Statistic animations

- Modal transitions
- Sidebar transitions
- Loading skeletons
- Success feedback
- Sensor transitions

Do not overuse animation.
Support prefers-reduced-motion where possible.

19. AUTH PAGES

Create attractive responsive Login and Register pages/modals matching ParkEase branding.

20. NAVIGATION

Logged out:

Home, Parking, About, Login, Register

Logged in:

Home, Parking, My Booking, My Account, Logout

Admin gets a separate interface.

21. AUTHORIZATION

Roles:

USER

ADMIN

Backend must enforce authorization.

Normal users cannot access admin APIs/pages.

Do not trust localStorage, hidden buttons, URL parameters, or frontend role values.

22. API

Use clean endpoints, for example:

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

GET /api/auth/me

Admin:

POST /api/admin/login

POST /api/admin/logout

GET /api/admin/dashboard

GET /api/admin/bookings

GET /api/admin/vehicles

GET /api/admin/statistics

GET /api/admin/iot-status

Use existing conventions where appropriate.

23. DATABASE

Inspect current models first.

Potential entities:

User

Booking

ParkingSlot

ParkingSystemState

Use safe migrations/schema updates.

Do not delete useful data during normal startup.

Do not create public admin registration.

24. LOCAL STORAGE

LocalStorage may be used for theme/UI preferences.

It must NOT be the source of truth for:

- Authentication
- Booking ownership
- Slot availability
- Parking state
- Admin authorization

25. KNOWN ISSUES

Previously observed:

- stale localStorage booking
- movement stuck ENTERING
- need for entry-crossed
- slot reports arriving after movement becomes IDLE
- slot staying RESERVED instead of OCCUPIED
- booking showing "Unable to create booking"

Do not guess the cause.

Inspect schema, models, constraints, transactions, logs, imports and actual running code.

Fix root causes.

26. RUNTIME CHECK

Verify that the code being edited is the code actually executed.

Check:

- project structure
- Flask entry point
- imports
- virtual environment
- requirements
- .env
- database URL
- migrations
- running process
- frontend API base URL

27. ENVIRONMENT VARIABLES

Use:

DATABASE_URL

SECRET_KEY

IOT_API_KEY

ADMIN_EMAIL

ADMIN_PASSWORD or ADMIN_PASSWORD_HASH

FRONTEND_URL

Never commit secrets.

Never expose secrets in frontend code.

28. CORS

Configure CORS for the real Vercel frontend origin and local development origin.

Avoid unrestricted wildcard origins when authenticated credentials/cookies are used.

29. DEPLOYMENT

Target:

Frontend → Vercel

Backend → Render or Railway

Database → Hosted MySQL

ESP32 → deployed backend API

Do not hardcode localhost, 127.0.0.1 or 10.81.231.98 into production logic. Use environment variables.

30. ESP32

Do not unnecessarily modify the existing firmware.

Maintain all GPIO assignments, active-low logic, gate behavior, authorization polling, exit authorization, slot reporting and entry-crossed reporting.

Only change ESP32 code when required by backend API changes.

31. USER EXPERIENCE

Use clear progress/status messages and toast notifications.

Never leave users unsure whether an action succeeded.

32. ERROR HANDLING

Examples:

Please log in to continue.

Invalid email or password.

Slot S1 is no longer available.

You already have an active booking.

Your booking is not eligible for entry.

Vehicle not detected at entry.

Unable to connect to parking server.

Admin authentication required.

Keep detailed technical errors in backend logs and safe friendly errors in the UI.

33. LOADING STATES

Add loading states and prevent double-clicking for login, registration, booking, cancellation, entry, admin data, statistics and live status.

34. ACCESSIBILITY

Use semantic HTML, labels, keyboard navigation, focus states, good contrast, aria-labels where needed, and reduced-motion support.

Do not rely only on color for status.

35. PERFORMANCE

Keep frontend fast, animations smooth, API requests reasonable and ESP32 requests lightweight.

Do not spam the backend.

36. CODE QUALITY

Understand the existing project before changes.

Remove dead/duplicate code.

Keep naming/imports consistent.
Validate inputs.
Comment complex state-machine logic.
Do not create competing implementations.

37. PRESERVE EXISTING UI

Keep ParkEase and Smart Parking branding.
Improve rather than completely replace the project's identity.
Do not break existing working features.

38. TESTING

AUTH:

- Register
- Duplicate email rejection
- Login
- Wrong password
- Logout
- Protected pages
- User cannot access admin

ADMIN:

- Admin login
- Wrong credentials
- Dashboard
- Live slots
- Bookings
- Active vehicles
- Statistics
- Sensors
- Gate
- Logout

BOOKING:

- Available slot booking
- RESERVED state
- Concurrent booking prevention
- User sees only own booking
- Cancellation

ENTRY:

- Entry IR
- Enter Parking
- Backend validation
- ESP32 authorization
- Servo opens
- Exit IR
- Entry-crossed
- Movement IDLE
- Gate closes
- Slot OCCUPIED

EXIT:

- Exit IR
- Exit authorization
- Slot EXITING
- Gate opens
- Entry IR completion
- EXITING → AVAILABLE
- Gate closes

RESPONSIVE:

- 320px
- 375px
- 390px
- 430px
- Tablet
- Laptop
- Desktop

39. ACTUAL REPOSITORY INSPECTION

Before implementation:

1. Inspect all files.
2. Identify frontend entry files.
3. Identify Flask entry point.
4. Identify models.
5. Identify services.
6. Identify routes.
7. Identify database configuration.
8. Identify authentication.
9. Identify ESP32 code.
10. Identify CSS/JS.

11. Identify environment variables.
12. Identify deployment configuration.

Do not assume the repository exactly matches this prompt.
Preserve working functionality and intelligently integrate requested features.

40. DO NOT ASK ME TO MANUALLY EDIT MANY FILES

Inspect the repository, make coherent changes, update all affected files, keep imports/API contracts consistent and ensure frontend/backend compatibility.
When practical, provide complete updated files or make changes directly in the repository.

41. FINAL DELIVERABLE

USER APPLICATION:

- Modern landing page
- Responsive design
- Register/login/logout
- Live parking
- Booking
- My Booking
- Enter Parking
- Cancel booking
- User account
- Animations
- Mobile responsive UI

ADMIN APPLICATION:

- Admin login
- Admin dashboard
- Live parking
- All bookings
- Active vehicles
- Statistics
- Sensor status
- Gate status
- Movement state
- Responsive interface

BACKEND:

- Flask REST API
- Authentication
- Authorization
- Secure password hashing
- User management
- Booking management
- Parking state management
- IoT APIs
- Admin APIs
- Transactions
- Concurrency-safe booking
- CORS
- Environment configuration

DATABASE:

- Users
- Bookings
- Parking slots
- Parking system state
- Constraints/indexes
- Safe migration

IOT:

- ESP32
- Entry/Exit IR
- S1/S2/S3 sensors
- Servo gate
- Backend communication
- Entry/exit authorization
- Slot occupancy updates

DEPLOYMENT:

Frontend → Vercel
Backend → Render or Railway
Database → Hosted MySQL
ESP32 → deployed backend API

42. MOST IMPORTANT RULE

DO NOT SACRIFICE EXISTING WORKING PARKING/ESP32 LOGIC WHILE ADDING NEW FEATURES.

Everything must use the same backend/database state.

43. FINAL REPORT

After implementation, provide:

1. Files changed
2. Features added
3. Database changes
4. New APIs
5. Authentication approach
6. Admin login configuration
7. Frontend improvements
8. ESP32 changes
9. Environment variables
10. Local run commands
11. Vercel deployment steps
12. Render/Railway deployment steps
13. Database setup/migration
14. Testing results
15. Remaining issues

Do not claim something works unless verified.
If something cannot be tested, clearly say so.

Priority:

1. Correctness
2. Existing ESP32/parking functionality
3. Security/authentication
4. Database consistency
5. Admin functionality
6. Responsive UI
7. Visual polish/animations
8. Deployment readiness

Start by inspecting the repository. Do not start by rewriting the project.